

CEUB

CURSO DE ENGENHARIA DE COMPUTAÇÃO

DISCIPLINA: Desenvolvimento Web

SISTEMA DE GESTÃO PARA ADOÇÃO DE PETS E ARRECADAÇÃO COMERCIAL

Entrega 1 – Modelagem e Arquitetura

Integrantes:
Rodrigo Pepato

Professor: Felipe Pires Ferreira

Brasília – Distrito Federal
2026

1. Introdução

Este documento apresenta a modelagem e arquitetura do Sistema de Gestão para Adoção de Pets e Arrecadação Comercial, desenvolvido para a disciplina de desenvolvimento de sistemas. O trabalho tem como finalidade aplicar conceitos de arquitetura em camadas, modelagem orientada a objetos, banco de dados relacional, API REST e autenticação JWT no desenvolvimento de uma aplicação web.

2. Descrição do Sistema

O sistema tem como objetivo auxiliar ONGs e instituições de proteção animal no gerenciamento de adoções de pets e no controle comercial de produtos vendidos para arrecadação de recursos.

A aplicação permitirá o cadastro de clientes/adotantes, pets disponíveis para adoção, produtos comercializados, usuários do sistema, registro de vendas, controle de estoque e geração de relatórios administrativos.

O sistema possuirá autenticação por token JWT e controle de acesso baseado em perfis de usuário.

3. Requisitos Funcionais

RF01 — O sistema deve permitir autenticação de usuários através de login e senha.

RF02 — O sistema deve permitir cadastrar, editar, listar e remover clientes/adotantes.

RF03 — O sistema deve permitir cadastrar, editar, listar e remover pets disponíveis para adoção.

RF04 — O sistema deve permitir cadastrar, editar, listar e remover produtos comercializados pela ONG.

RF05 — O sistema deve controlar automaticamente o estoque dos produtos.

RF06 — O sistema deve permitir registrar vendas de produtos.

RF07 — O sistema deve calcular automaticamente o valor total da venda.

RF08 — O sistema deve impedir vendas com estoque insuficiente.

RF09 — O sistema deve permitir registrar adoções de pets.

RF10 — O sistema deve permitir gerar relatórios de vendas por período, cliente e produto.

RF11 — O sistema deve possuir controle de acesso baseado em perfil de usuário (ADMIN e FUNCIONARIO).

RF12 — O sistema deve disponibilizar API REST para comunicação com a interface web.

4. Requisitos Não Funcionais

RNF01 — O sistema deve utilizar arquitetura em camadas.

RNF02 — O sistema deve utilizar banco de dados MySQL para persistência das informações.

RNF03 — A API deve retornar respostas no formato JSON.

RNF04 — O sistema deve possuir autenticação utilizando JWT.

RNF05 — As senhas dos usuários devem ser armazenadas de forma criptografada.

RNF06 — O sistema deve possuir controle de acesso por perfil de usuário.

RNF07 — O sistema deve ser desenvolvido utilizando versionamento no GitHub.

RNF08 — O sistema deve possuir interface web responsiva.

RNF09 — O sistema deve apresentar boa organização e legibilidade de código.

RNF10 — O sistema deve permitir manutenção e expansão futuras.

5. Arquitetura Proposta

O sistema será desenvolvido utilizando arquitetura em camadas, separando as responsabilidades da aplicação em diferentes níveis para facilitar manutenção, organização e escalabilidade.

A aplicação será composta por:

- Camada de Interface Web (Django)
Responsável pela interação com o usuário e consumo da API REST.
- Camada de API REST
Responsável pelas regras de negócio, autenticação JWT e comunicação com o banco de dados.
- Camada de Persistência
Responsável pelo acesso e manipulação das informações no banco de dados.
- Banco de Dados MySQL
Responsável pelo armazenamento persistente das informações do sistema, garantindo integridade e organização dos dados.

6. Padrões de Projeto

O sistema utilizará alguns padrões de projeto e organização de software para melhorar a manutenção, reutilização e organização do código.

- Arquitetura em Camadas
Separação entre interface, regras de negócio e persistência de dados.
- Repository Pattern
Responsável por centralizar o acesso aos dados do banco de dados.
- MVC (Model-View-Controller)
Utilizado na organização da interface web desenvolvida em Django.
- REST API
Padronização da comunicação entre frontend e backend utilizando requisições HTTP e respostas em JSON.
- JWT (JSON Web Token)
Utilizado para autenticação e autorização de usuários no sistema.